# CSE 816: Software Production Engineering

**Mini-Project (10 Marks): Problem Statement**

*End-to-End CI/CD Pipeline for a Calculator REST API*

Term I, 2026–27 | Stack: Java (Spring Boot + Maven)

**Faculty: Prof. B. Thangaraju**

| Item | Project details |
|---|---|
| **Student name** | Kaushalraj Puwar |
| **Roll number** | IMT2023063 |
| **GitHub repository** | github.com/KaushalrajPuwar/arithmos-pipeline |
| **Docker Hub image** | hub.docker.com/r/kaushalrajpuwar/arithmos-calculator (public; tag `latest`) |
| **Webhook domain** | unstaffed-disperser-tweezers.ngrok-free.dev (endpoint: `/github-webhook/`) |
| **Technology stack** | Java 21, Spring Boot, Maven, JUnit 5, Jenkins, GitHub, Docker, Compose, Nginx, ngrok |
| **Submission date** | October 30, 2026 |

## 1. Project objective

Build a small, production-style Calculator REST API in Java and connect it to an automated delivery pipeline. A GitHub push should trigger Jenkins to compile and test the application, package and publish a Docker image, deploy the Compose stack, and verify service health. The assessment emphasizes DevOps engineering rather than mathematical complexity. The scope ends at Compose-based deployment.

## 2. Application and verification

### REST API

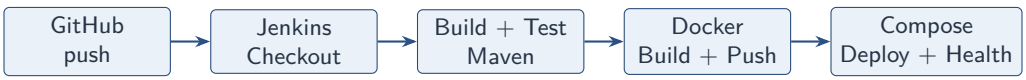
| Operation | Endpoint | Response / behaviour |
|---|---|---|
| **Add** | `/api/v1/add?a=&b=` | Returns `a + b` as JSON. |
| **Subtract** | `/api/v1/subtract?a=&b=` | Returns `a - b` as JSON. |
| **Multiply** | `/api/v1/multiply?a=&b=` | Returns `a * b` as JSON. |
| **Divide** | `/api/v1/divide?a=&b=` | Returns `a / b`; zero divisor gives HTTP 400 and structured error JSON. |
| **Health** | `/actuator/health` | Spring Actuator UP/DOWN probe used by Compose. |

Successful calculations return `operation`, `a`, `b`, and `result`. Missing or non-numeric inputs and division by zero return HTTP 400 with timestamp, status, message, and request path. The Spring Boot application is Maven-based and produces an executable JAR.

### Test coverage

The suite contains 8 JUnit 5 service tests and 7 MockMvc tests covering all four operations, negative and decimal values, missing/invalid inputs, and division by zero. `mvn test` produces Surefire JUnit XML; Jenkins publishes those reports and fails a build on test failures. Final local run: 15 tests, 0 failures, 0 errors.

## 3. Delivery flow




The declarative Jenkinsfile runs six stages: Checkout, Build (`mvn clean package -DskipTests`), Test (`mvn test`), Docker Build (build-number and `latest` tags), Push (Jenkins-managed Docker Hub credential), and Deploy. The repository is configured for GitHub push events through the reserved ngrok domain; `githubPush()` is the Jenkins trigger. The webhook endpoint is `https://unstaffed-disperser-tweezers.ngrok-free.dev/github-webhook/`. Build 10 was verified with cause "Started by GitHub push by KaushalrajPuwar"; the GitHub push delivery returned HTTP 200. The tunnel must be running for GitHub to reach the locally hosted Jenkins instance. For a later demo, run `brew services start jenkins-lts`, then start `ngrok http 8080 -url https://unstaffed-disperser-tweezers.ngrok-free.dev` in a terminal and leave it open.

## 4. Containerization, deployment and evidence

The multi-stage Dockerfile compiles the JAR in a Maven/JDK 21 build stage, then copies only the artifact into a slim JRE Alpine runtime. The runtime uses a non-root app account and exposes port 8080. The image is available publicly as `kaushalrajpuwar/arithmos-calculator` on Docker Hub.

Docker Compose starts two services on a shared network: the calculator API (host port 8081) and an Nginx reverse proxy (host port 8082). The app has an Actuator health check, restart policy, and configurable port environment variable; Nginx waits for the app to become healthy. A clean-slate Compose run and a standalone `docker run` were both verified.

### Pipeline stages and deployment checks

| Stage | Automated action |
|---|---|
| **Checkout** | Fetch the triggered revision from GitHub. |
| **Build and Test** | Package with Maven, run JUnit and MockMvc tests, publish XML results. |
| **Docker Build and Push** | Build build-number and `latest` tags, authenticate using Jenkins credentials, publish to Docker Hub. |
| **Deploy** | Recreate the Compose stack, wait for health, verify an API request through Nginx. |

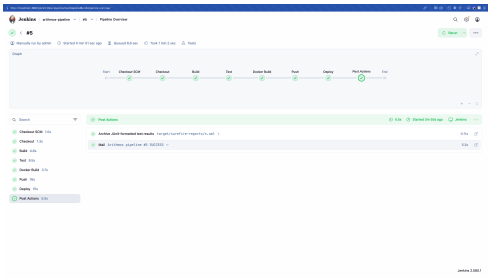


**Figure 1:** Successful Jenkins run: all stages and post actions green.

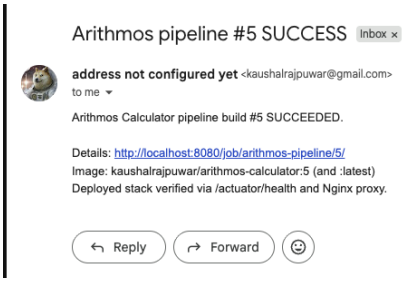



**Figure 2:** Build-success email notification.

```
❯ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                STATUS                    PORTS
arithmos-proxy       Up 4 minutes              0.0.0.0:8082→80/tcp
arithmos-calculator  Up 5 minutes (healthy)    0.0.0.0:8081→8080/tcp
```

**Figure 3:** Compose services running; calculator healthy.

```
kaushalrajpuwar ~ 20:31
❯ curl "http://localhost:8082/api/v1/add?a=2&b=3"
{"operation":"add","a":2.0,"b":3.0,"result":5.0}%

kaushalrajpuwar ~ 20:32
❯ curl "http://localhost:8082/api/v1/divide?a=5&b=0"
{"timestamp":"2026-10-30T15:02:20.4540348457","status":400,"error":"Bad Request","messag
e":"Division by zero is not allowed","path":"/api/v1/divide"}%

kaushalrajpuwar ~ 20:32
❯ curl http://localhost:8081/actuator/health
{"status":"UP","groups":["liveness","readiness"]}%
```

**Figure 4:** API response, divide-by-zero error, and health probe.

## 5. Challenges faced and solutions

- **JDK mismatch in Jenkins.** The machine default resolved to Java 27 although the project targets Java 21; Mockito inline mocks failed in that runtime. *Resolution:* pinned `JAVA_HOME` and `PATH` to the installed Java 21 runtime; the full suite then passed.
- **Compose container-name collision.** Manual test containers remained when Jenkins deployed from a different workspace. *Resolution:* made deployment repeatable by running `docker compose down -remove-orphans` before `up -d`, followed by a health and proxy check.
- **Public webhook reachability and signature validation.** GitHub cannot deliver to localhost, and Jenkins initially rejected webhook signatures because the secret was not referenced as a Jenkins credential. *Resolution:* used the reserved ngrok domain, stored a dedicated secret in Jenkins Credentials, selected SHA-256 verification, and confirmed GitHub ping/push deliveries return HTTP 200.
- **Email delivery.** Jenkins initially attempted an unavailable local SMTP service. *Resolution:* configured Gmail SMTP with TLS and stored the app password in Jenkins configuration, then verified the success notification in the inbox.

## 6. Conclusion

The project delivers a working, repeatable path from a GitHub push to a tested release and health-verified deployment. The Calculator API has four operations, structured input errors, and an Actuator probe. Its 15-test suite runs in Jenkins, which builds and publishes versioned Docker images, deploys the two-service Compose stack, verifies the API through Nginx, and sends build notifications. GitHub push events reach Jenkins through the configured ngrok webhook. The public source and image remain available at the links below.

### Deliverables tally

| Deliverable | Status / evidence |
|---|---|
| **Spring Boot API and tests** | Complete; 15 tests pass, JUnit XML published by Jenkins. |
| **GitHub repository and history** | Complete; public repository with feature branch and merged pull request. |
| **Jenkins CI/CD and webhook** | Complete; webhook-triggered build number 10 succeeded, email notification verified. |
| **Docker image and Compose deployment** | Complete; public image and healthy calculator plus Nginx services. |
| **Report evidence** | Complete; four screenshots included in this report. |

**Repository:** GitHub **Container image:** Docker Hub.